01 LC 1 | Two Sum (Easy)
Problem: Return the indices of two distinct elements whose sum is target. Exactly one pair exists; either index order is valid.
Constraints: 2 <= nums.length <= 10^4; -10^9 <= nums[i] <= 10^9; -10^9 <= target <= 10^9; Only one valid answer exists.

```
class Solution {
public:
  vector<int> twoSum(vector<int>& nums, int
    target) {
    unordered_map<int, int> d;
    for (int i = 0;; ++i) {
      int x = nums[i];
      int y = target - x;
      if (d.contains(y)) {
        return {d[y], i};
      }
      d[x] = i;
    }
  }
};
```

20 LC 14 | Longest Common Prefix (Easy)
Problem: Return the longest prefix shared by every string in strs, or an empty string if none exists.
Constraints: 1 <= strs.length <= 200; 0 <= strs[i].length <= 200; strs[i] consists of only lowercase English letters if it is non-empty.

```
class Solution {
public:
  string longestCommonPrefix(vector<string>& strs)
    {
    int n = strs.size();
    for (int i = 0; i < strs[0].size(); ++i) {
      for (int j = 1; j < n; ++j) {
        if (strs[j].size() <= i || strs[j][i] !=
          strs[0][i]) {
          return strs[0].substr(0, i);
        }
      }
    }
    return strs[0];
  }
};
```

34 LC 48 | Rotate Image (Medium)
Problem: Rotate the n-by-n matrix 90 degrees clockwise in place. Do not allocate another matrix.
Constraints: n == matrix.length == matrix[i].length; 1 <= n <= 20; -1000 <= matrix[i][j] <= 1000

```
class Solution {
public:
  void rotate(vector<vector<int>>& matrix) {
    int n = matrix.size();
    for (int i = 0; i < n >> 1; ++i) {
      for (int j = 0; j < n; ++j) {
        swap(matrix[i][j], matrix[n - i - 1][j]);
      }
    }
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < i; ++j) {
        swap(matrix[i][j], matrix[j][i]);
      }
    }
  }
};
```

02 LC 3 | Longest Substring Without Repeating Characters (Medium)
Problem: Return the length of the longest contiguous substring of s containing no repeated characters.
Constraints: 0 <= s.length <= 10^5; s consists of English letters, digits, symbols and spaces.

```
class Solution {
public:
  int lengthOfLongestSubstring(string s) {
    int cnt[128]{};
    int ans = 0, n = s.size();
    for (int l = 0, r = 0; r < n; ++r) {
      ++cnt[s[r]];
      while (cnt[s[r]] > 1) {
        --cnt[s[l++]];
      }
      ans = max(ans, r - l + 1);
    }
    return ans;
  }
};
```

11 LC 56 | Merge Intervals (Medium)
Problem: Merge all overlapping closed intervals and return nonoverlapping intervals covering exactly the same points.
Constraints: 1 <= intervals.length <= 10^4; intervals[i].length == 2; 0 <= start_i <= end_i <= 10^4

```
class Solution {
public:
  vector<vector<int>> merge(vector<vector<int>>&
    intervals) {
    ranges::sort(intervals);
    int st = intervals[0][0], ed =
      intervals[0][1];
    vector<vector<int>> ans;
    for (int i = 1; i < intervals.size(); ++i) {
      if (ed < intervals[i][0]) {
        ans.push_back({st, ed});
        st = intervals[i][0];
        ed = intervals[i][1];
      } else {
        ed = max(ed, intervals[i][1]);
      }
    }
    ans.push_back({st, ed});
    return ans;
  }
};
```

18 LC 125 | Valid Palindrome (Easy)
Problem: Ignore non-alphanumeric ASCII characters and letter case. Return whether the remaining characters read identically forward and backward.
Constraints: 1 <= s.length <= 2 * 10^5; s consists only of printable ASCII characters.

```
class Solution {
public:
  bool isPalindrome(string s) {
    int i = 0, j = s.size() - 1;
    while (i < j) {
      if (!isalnum(s[i])) {
        ++i;
      } else if (!isalnum(s[j])) {
        --j;
      } else if (tolower(s[i]) != tolower(s[j])) {
        return false;
      } else {
        ++i;
        --j;
      }
    }
    return true;
  }
};
```

03 LC 238 | Product of Array Except Self (Medium)
Problem: Return answer[i], the product of all nums elements except nums[i]. Do not use division. Required runtime is O(n); every prefix, suffix, and output product fits a signed 32-bit integer.
Constraints: 2 <= nums.length <= 10^5; -30 <= nums[i] <= 30; The input is generated such that answer[i] is guaranteed to fit in a 32-bit integer.

```
class Solution {
public:
  vector<int> productExceptSelf(vector<int>& nums)
    {
    int n = nums.size();
    vector<int> ans(n);
    for (int i = 0, left = 1; i < n; ++i) {
      ans[i] = left;
      left *= nums[i];
    }
    for (int i = n - 1, right = 1; ~i; --i) {
      ans[i] *= right;
      right *= nums[i];
    }
    return ans;
  }
};
```

28 LC 209 | Minimum Size Subarray Sum (Medium)
Problem: Return the minimum length of a nonempty contiguous subarray of positive nums whose sum is at least target, or 0 if no such subarray exists.
Constraints: 1 <= target <= 10^9; 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^4

```
class Solution {
public:
  int minSubArrayLen(int target, vector<int>&
    nums) {
    int l = 0, n = nums.size();
    long long s = 0;
    int ans = n + 1;
    for (int r = 0; r < n; ++r) {
      s += nums[r];
      while (s >= target) {
        ans = min(ans, r - l + 1);
        s -= nums[l++];
      }
    }
    return ans > n ? 0 : ans;
  }
};
```

33 LC 31 | Next Permutation (Medium)
Problem: Replace nums in place by its next lexicographically greater permutation. If it is already the greatest permutation, rearrange it into nondecreasing order. Use O(1) extra memory.
Constraints: 1 <= nums.length <= 100; 0 <= nums[i] <= 100

```
class Solution {
public:
  void nextPermutation(vector<int>& nums) {
    int n = nums.size();
    int i = n - 2;
    while (~i && nums[i] >= nums[i + 1]) {
      --i;
    }
    if (~i) {
      for (int j = n - 1; j > i; --j) {
        if (nums[j] > nums[i]) {
          swap(nums[i], nums[j]);
          break;
        }
      }
    }
    reverse(nums.begin() + i + 1, nums.end());
  }
};
```

04 LC 53 | Maximum Subarray (Medium)
Problem: Return the largest sum of a nonempty contiguous subarray of nums.
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int maxSubArray(vector<int>& nums) {
    int ans = nums[0], f = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
      f = max(f, 0) + nums[i];
      ans = max(ans, f);
    }
    return ans;
  }
};
```

07 LC 283 | Move Zeroes (Easy)
Problem: Move all zeros to the end of nums in place, preserving the relative order of nonzero elements. Do not copy the array.
Constraints: 1 <= nums.length <= 10^4; -2^31 <= nums[i] <= 2^31 - 1

```
class Solution {
public:
  void moveZeroes(vector<int>& nums) {
    int k = 0, n = nums.size();
    for (int i = 0; i < n; ++i) {
      if (nums[i]) {
        swap(nums[i], nums[k++]);
      }
    }
  }
};
```

13 LC 189 | Rotate Array (Medium)
Problem: Rotate nums in place to the right by k positions, with positions wrapping around the array.
Constraints: 1 <= nums.length <= 10^5; -2^31 <= nums[i] <= 2^31 - 1; 0 <= k <= 10^5

```
class Solution {
public:
  void rotate(vector<int>& nums, int k) {
    int n = nums.size();
    k %= n;
    ranges::reverse(nums);
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
  }
};
```

40 LC 55 | Jump Game (Medium)
Problem: Starting at index 0, nums[i] is the maximum forward jump length from i. Return whether the last index can be reached.
Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 10^5

```
class Solution {
public:
  bool canJump(vector<int>& nums) {
    int mx = 0;
    for (int i = 0; i < nums.size(); ++i) {
      if (mx < i) {
        return false;
      }
      mx = max(mx, i + nums[i]);
    }
    return true;
  }
};
```

05 LC 121 | Best Time to Buy and Sell Stock (Easy)
Problem: Choose one day to buy and a later day to sell one stock. Return the maximum profit, or 0 when no profitable transaction exists.
Constraints: 1 <= prices.length <= 10^5; 0 <= prices[i] <= 10^4

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int ans = 0, mi = prices[0];
    for (int& v : prices) {
      ans = max(ans, v - mi);
      mi = min(mi, v);
    }
    return ans;
  }
};
```

29 LC 239 | Sliding Window Maximum (Hard)
Problem: Slide a window of k consecutive elements across nums, advancing one position at a time. Return the maximum value in each window, in window order.
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4; 1 <= k <= nums.length

```
class Solution {
public:
  vector<int> maxSlidingWindow(vector<int>& nums,
    int k) {
    deque<int> q;
    vector<int> ans;
    for (int i = 0; i < nums.size(); ++i) {
      if (q.size() && i - q.front() >= k) {
        q.pop_front();
      }
      while (q.size() && nums[q.back()] <=
        nums[i]) {
        q.pop_back();
      }
      q.push_back(i);
      if (i >= k - 1) {
        ans.push_back(nums[q.front()]);
      }
    }
    return ans;
  }
};
```

39 LC 134 | Gas Station (Medium)
Problem: Stations form a circular route. Collect gas[i] at station i and spend cost[i] to reach the next station. Starting with an empty, unlimited tank, return a starting index that completes the circuit, or -1. A feasible start is unique.
Constraints: n == gas.length == cost.length; 1 <= n <= 10^5; 0 <= gas[i], cost[i] <= 10^4; The input is generated such that the answer is unique.

```
class Solution {
public:
  int canCompleteCircuit(vector<int>& gas,
    vector<int>& cost) {
    int n = gas.size();
    int i = n - 1, j = n - 1;
    int cnt = 0, s = 0;
    while (cnt < n) {
      s += gas[j] - cost[j];
      ++cnt;
      j = (j + 1) % n;
      while (s < 0 && cnt < n) {
        --i;
        s += gas[i] - cost[i];
        ++cnt;
      }
    }
    return s < 0 ? -1 : i;
  }
};
```

06 LC 26 | Remove Duplicates from Sorted Array (Easy)
Problem: Remove duplicates from sorted nums in place, preserving order. Return k and store the k distinct values in nums[0..k-1]; values after k do not matter.
Constraints: 1 <= nums.length <= 3 * 10^4; -100 <= nums[i] <= 100; nums is sorted in non-decreasing order.

```
class Solution {
public:
  int removeDuplicates(vector<int>& nums) {
    int k = 0;
    for (int x : nums) {
      if (k == 0 || x != nums[k - 1]) {
        nums[k++] = x;
      }
    }
    return k;
  }
};
```

30 LC 560 | Subarray Sum Equals K (Medium)
Problem: Return the number of nonempty contiguous subarrays of nums whose sum equals k.
Constraints: 1 <= nums.length <= 2 * 10^4; -1000 <= nums[i] <= 1000; -10^7 <= k <= 10^7

```
class Solution {
public:
  int subarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> cnt{{0, 1}};
    int ans = 0, s = 0;
    for (int x : nums) {
      s += x;
      ans += cnt[s - k];
      ++cnt[s];
    }
    return ans;
  }
};
```

46 LC 27 | Remove Element (Easy)
Problem: Remove all occurrences of val from nums in place. Return k and store the retained elements in nums[0..k-1]. Their order may change; entries after k do not matter.
Constraints: 0 <= nums.length <= 100; 0 <= nums[i] <= 50; 0 <= val <= 100

```
class Solution {
public:
  int removeElement(vector<int>& nums, int val) {
    int k = 0;
    for (int x : nums) {
      if (x != val) {
        nums[k++] = x;
      }
    }
    return k;
  }
};
```

47 LC 80 | Remove Duplicates from Sorted Array II (Medium)
Problem: Remove duplicates from sorted nums in place so each value appears at most twice, preserving order. Return k and store the retained values in nums[0..k-1]. Use O(1) extra memory.
Constraints: 1 <= nums.length <= 3 * 10^4; -10^4 <= nums[i] <= 10^4; nums is sorted in non-decreasing order.

```
class Solution {
public:
  int removeDuplicates(vector<int>& nums) {
    int k = 0;
    for (int x : nums) {
      if (k < 2 || x != nums[k - 2]) {
        nums[k++] = x;
      }
    }
    return k;
  }
};
```

08 LC 15 | 3Sum (Medium)

Problem: Return all distinct value triplets drawn from three different indices whose sum is zero. Do not return duplicate triplets.
Constraints: 3 <= nums.length <= 3000; -10^5 <= nums[i] <= 10^5

```
class Solution {
public:
  vector<vector<int>> threeSum(vector<int>& nums)
    {
    ranges::sort(nums);
    vector<vector<int>> ans;
    int n = nums.size();
    for (int i = 0; i < n - 2 && nums[i] <= 0;
      ++i) {
      if (i && nums[i] == nums[i - 1]) {
        continue;
      }
      int j = i + 1, k = n - 1;
      while (j < k) {
        int x = nums[i] + nums[j] + nums[k];
        if (x < 0) {
          ++j;
        } else if (x > 0) {
          --k;
        } else {
          ans.push_back({nums[i], nums[j++],
            nums[k--]});
          while (j < k && nums[j] == nums[j - 1])
            {
            ++j;
          }
          while (j < k && nums[k] == nums[k + 1])
            {
            --k;
          }
        }
      }
    }
    return ans;
  }
};
```

35 LC 54 | Spiral Matrix (Medium)

Problem: Return the elements of matrix in clockwise spiral order, starting at the top-left corner.
Constraints: m == matrix.length; n == matrix[i].length; 1 <= m, n <= 10; -100 <= matrix[i][j] <= 100

```
class Solution {
public:
  vector<int> spiralOrder(vector<vector<int>>&
    matrix) {
    int m = matrix.size(), n = matrix[0].size();
    int dirs[5] = {0, 1, 0, -1, 0};
    int i = 0, j = 0, k = 0;
    vector<int> ans;
    for (int h = m * n; h; --h) {
      ans.push_back(matrix[i][j]);
      matrix[i][j] += 300;
      int x = i + dirs[k], y = j + dirs[k + 1];
      if (x < 0 || x >= m || y < 0 || y >= n ||
        matrix[x][y] > 100) {
        k = (k + 1) % 4;
      }
      i += dirs[k];
      j += dirs[k + 1];
    }
    for (i = 0; i < m; ++i) {
      for (j = 0; j < n; ++j) {
        matrix[i][j] -= 300;
      }
    }
    return ans;
  }
};
```

09 LC 11 | Container With Most Water (Medium)

Problem: Vertical lines at indices i have heights height[i]. Choose two lines that form an upright container with the x-axis; return its maximum water capacity.
Constraints: n == height.length; 2 <= n <= 10^5; 0 <= height[i] <= 10^4

```
class Solution {
public:
  int maxArea(vector<int>& height) {
    int l = 0, r = height.size() - 1;
    int ans = 0;
    while (l < r) {
      int t = min(height[l], height[r]) * (r - l);
      ans = max(ans, t);
      if (height[l] < height[r]) {
        ++l;
      } else {
        --r;
      }
    }
    return ans;
  }
};
```

36 LC 73 | Set Matrix Zeroes (Medium)

Problem: For every zero in the original matrix, set its entire row and column to zero. Modify the matrix in place.
Constraints: m == matrix.length; n == matrix[0].length; 1 <= m, n <= 200; -2^31 <= matrix[i][j] <= 2^31 - 1

```
class Solution {
public:
  void setZeroes(vector<vector<int>>& matrix) {
    int m = matrix.size(), n = matrix[0].size();
    bool i0 = false, j0 = false;
    for (int j = 0; j < n; ++j) {
      if (matrix[0][j] == 0) {
        i0 = true;
        break;
      }
    }
    for (int i = 0; i < m; ++i) {
      if (matrix[i][0] == 0) {
        j0 = true;
        break;
      }
    }
    for (int i = 1; i < m; ++i) {
      for (int j = 1; j < n; ++j) {
        if (matrix[i][j] == 0) {
          matrix[i][0] = 0;
          matrix[0][j] = 0;
        }
      }
    }
    for (int i = 1; i < m; ++i) {
      for (int j = 1; j < n; ++j) {
        if (matrix[i][0] == 0 || matrix[0][j] ==
          0) {
          matrix[i][j] = 0;
        }
      }
    }
    if (i0) {
      for (int j = 0; j < n; ++j) {
        matrix[0][j] = 0;
      }
    }
    if (j0) {
      for (int i = 0; i < m; ++i) {
        matrix[i][0] = 0;
      }
    }
  }
};
```

10 LC 42 | Trapping Rain Water (Hard)

Problem: Bars have heights height[i] and unit width. Return the total water trapped between the bars after rain.

Constraints: n == height.length; 1 <= n <= 2 * 10^4; 0 <= height[i] <= 10^5

```
class Solution {
public:
  int trap(vector<int>& height) {
    int left = 0, right =
      static_cast<int>(height.size()) - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left <= right) {
      if (height[left] <= height[right]) {
        leftMax = max(leftMax, height[left]);
        water += leftMax - height[left++];
      } else {
        rightMax = max(rightMax, height[right]);
        water += rightMax - height[right--];
      }
    }
    return water;
  }
};
```

44 LC 18 | 4Sum (Medium)

Problem: Return every distinct value quadruplet from four different indices whose sum equals target. Output order does not matter; duplicate quadruplets are forbidden.

Constraints: 1 <= nums.length <= 200; -10^9 <= nums[i] <= 10^9; -10^9 <= target <= 10^9

```
class Solution {
public:
  vector<vector<int>> fourSum(vector<int>& nums,
    int target) {
    int n = nums.size();
    vector<vector<int>> ans;
    if (n < 4) {
      return ans;
    }
    ranges::sort(nums);
    for (int i = 0; i < n - 3; ++i) {
      if (i && nums[i] == nums[i - 1]) {
        continue;
      }
      for (int j = i + 1; j < n - 2; ++j) {
        if (j > i + 1 && nums[j] == nums[j - 1]) {
          continue;
        }
        int k = j + 1, l = n - 1;
        while (k < l) {
          long long x = (long long) nums[i] +
            nums[j] + nums[k] + nums[l];
          if (x < target) {
            ++k;
          } else if (x > target) {
            --l;
          } else {
            ans.push_back({nums[i], nums[j],
              nums[k++], nums[l--]});
            while (k < l && nums[k] == nums[k -
              1]) {
              ++k;
            }
            while (k < l && nums[l] == nums[l +
              1]) {
              --l;
            }
          }
        }
      }
    }
    return ans;
  }
};
```

12 LC 88 | Merge Sorted Array (Easy)

Problem: Merge sorted nums2 into sorted nums1 in place. nums1 has m valid entries followed by n spare slots; nums2 has n entries. The final m+n entries must be nondecreasing.

Constraints: nums1.length == m + n; nums2.length == n; 0 <= m, n <= 200; 1 <= m + n <= 200; -10^9 <= nums1[i], nums2[j] <= 10^9

```
class Solution {
public:
  void merge(vector<int>& nums1, int m,
    vector<int>& nums2, int n) {
    for (int i = m - 1, j = n - 1, k = m + n - 1;
      ~j; --k) {
      nums1[k] = i >= 0 && nums1[i] > nums2[j] ?
        nums1[i--] : nums2[j--];
    }
  }
};
```

43 LC 347 | Top K Frequent Elements (Medium)

Problem: Return the k most frequent distinct values in nums, in any order. The answer set is unique. Required runtime is better than O(n log n).

Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4; k is in the range [1, the number of unique elements in the array].; It is guaranteed that the answer is unique.

```
class Solution {
public:
  vector<int> topKFrequent(vector<int>& nums, int
    k) {
    unordered_map<int, int> frequency;
    for (int x : nums) ++frequency[x];
    vector<vector<int>> buckets(nums.size() + 1);
    for (auto [value, count] : frequency)
      buckets[count].push_back(value);
    vector<int> answer;
    for (int count = nums.size(); count > 0;
      --count)
      for (int value : buckets[count]) {
        answer.push_back(value);
        if (static_cast<int>(answer.size()) == k)
          return answer;
      }
    return answer;
  }
};
```

45 LC 16 | 3Sum Closest (Medium)

Problem: Choose three different indices whose values sum closest to target. Return that sum. The input guarantees exactly one solution.

Constraints: 3 <= nums.length <= 500; -1000 <= nums[i] <= 1000; -10^4 <= target <= 10^4

```
class Solution {
public:
  int threeSumClosest(vector<int>& nums, int
    target) {
    ranges::sort(nums);
    int ans = 1 << 30;
    int n = nums.size();
    for (int i = 0; i < n; ++i) {
      int j = i + 1, k = n - 1;
      while (j < k) {
        int t = nums[i] + nums[j] + nums[k];
        if (t == target) return t;
        if (abs(t - target) < abs(ans - target))
          ans = t;
        if (t > target)
          --k;
        else
          ++j;
      }
    }
    return ans;
  }
};
```

14 LC 75 | Sort Colors (Medium)
Problem: Sort nums in place so all 0s precede all 1s, followed by all 2s. Do not call a library sort function.
Constraints: n == nums.length; 1 <= n <= 300; nums[i] is either 0, 1, or 2.

```
class Solution {
public:
  void sortColors(vector<int>& nums) {
    int i = -1, j = nums.size(), k = 0;
    while (k < j) {
      if (nums[k] == 0) {
        swap(nums[++i], nums[k++]);
      } else if (nums[k] == 2) {
        swap(nums[--j], nums[k]);
      } else {
        ++k;
      }
    }
  }
};
```

16 LC 217 | Contains Duplicate (Easy)
Problem: Return true if any value occurs at least twice in nums; otherwise return false.
Constraints: 1 <= nums.length <= 10^5; -10^9 <= nums[i] <= 10^9

```
class Solution {
public:
  bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> s(nums.begin(),
      nums.end());
    return s.size() < nums.size();
  }
};
```

19 LC 242 | Valid Anagram (Easy)
Problem: Return whether s and t are anagrams: they contain exactly the same letters with the same multiplicities.
Constraints: 1 <= s.length, t.length <= 5 * 10^4; s and t consist of lowercase English letters.

```
class Solution {
public:
  bool isAnagram(string s, string t) {
    if (s.size() != t.size()) {
      return false;
    }
    vector<int> cnt(26);
    for (int i = 0; i < s.size(); ++i) {
      ++cnt[s[i] - 'a'];
      --cnt[t[i] - 'a'];
    }
    return all_of(cnt.begin(), cnt.end(), [](int
      x) { return x == 0; });
  }
};
```

41 LC 45 | Jump Game II (Medium)
Problem: Starting at index 0, jump forward by at most nums[i] positions from index i. Return the minimum jumps needed to reach the last index, which is guaranteed reachable.
Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 1000; It's guaranteed that you can reach nums[n - 1].

```
class Solution {
public:
  int jump(vector<int>& nums) {
    int ans = 0, mx = 0, last = 0;
    for (int i = 0; i < nums.size() - 1; ++i) {
      mx = max(mx, i + nums[i]);
      if (last == i) {
        ++ans;
        last = mx;
      }
    }
    return ans;
  }
};
```

15 LC 169 | Majority Element (Easy)
Problem: Return the value that occurs more than floor(n/2) times in nums. Such a majority value is guaranteed to exist.
Constraints: n == nums.length; 1 <= n <= 5 * 10^4; -10^9 <= nums[i] <= 10^9; The input is generated such that a majority element will exist in the array.

```
class Solution {
public:
  int majorityElement(vector<int>& nums) {
    int cnt = 0, m = 0;
    for (int& x : nums) {
      if (cnt == 0) {
        m = x;
        cnt = 1;
      } else {
        cnt += m == x ? 1 : -1;
      }
    }
    return m;
  }
};
```

32 LC 41 | First Missing Positive (Hard)
Problem: Return the smallest positive integer absent from unsorted nums. Required runtime is O(n), with O(1) auxiliary space.
Constraints: 1 <= nums.length <= 10^5; -2^31 <= nums[i] <= 2^31 - 1

```
class Solution {
public:
  int firstMissingPositive(vector<int>& nums) {
    int n = nums.size();
    for (int i = 0; i < n; ++i) {
      while (nums[i] > 0 && nums[i] <= n &&
        nums[i] != nums[nums[i] - 1]) {
        swap(nums[i], nums[nums[i] - 1]);
      }
    }
    for (int i = 0; i < n; ++i) {
      if (nums[i] != i + 1) {
        return i + 1;
      }
    }
    return n + 1;
  }
};
```

42 LC 128 | Longest Consecutive Sequence (Medium)
Problem: Return the length of the longest sequence of consecutive integer values present in unsorted nums. The values need not be adjacent in the array. Required runtime is O(n).
Constraints: 0 <= nums.length <= 10^5; -10^9 <= nums[i] <= 10^9

```
class Solution {
public:
  int longestConsecutive(vector<int>& nums) {
    unordered_set<int> s(nums.begin(),
      nums.end());
    int ans = 0;
    for (int x : s) {
      if (!s.contains(x - 1)) {
        int y = x + 1;
        while (s.contains(y)) {
          y++;
        }
        ans = max(ans, y - x);
      }
    }
    return ans;
  }
};
```

17 LC 49 | Group Anagrams (Medium)

Problem: Group strings that are anagrams: their characters have identical frequencies. Groups and their contents may be returned in any order.
Constraints: 1 <= strs.length <= 10^4; 0 <= strs[i].length <= 100; strs[i] consists of lowercase English letters.

```
class Solution {
public:
  vector<vector<string>>
    groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> d;
    for (auto& s : strs) {
      int cnt[26] = {0};
      for (auto& c : s) ++cnt[c - 'a'];
      string k;
      for (int i = 0; i < 26; ++i) {
        if (cnt[i]) {
          k += 'a' + i;
          k += to_string(cnt[i]);
        }
      }
      d[k].emplace_back(s);
    }
    vector<vector<string>> ans;
    for (auto& [_, v] : d) ans.emplace_back(v);
    return ans;
  }
};
```

22 LC 344 | Reverse String (Easy)

Problem: Reverse the character array s in place, using only O(1) extra memory.
Constraints: 1 <= s.length <= 10^5; s[i] is a printable ascii character.

```
class Solution {
public:
  void reverseString(vector<char>& s) {
    for (int i = 0, j = s.size() - 1; i < j;) {
      swap(s[i++], s[j--]);
    }
  }
};
```

26 LC 438 | Find All Anagrams in a String (Medium)

Problem: Return every starting index of a substring of s that is an anagram of p. Indices may be returned in any order.
Constraints: 1 <= s.length, p.length <= 3 * 10^4; s and p consist of lowercase English letters.

```
class Solution {
public:
  vector<int> findAnagrams(string s, string p) {
    int m = s.size(), n = p.size();
    vector<int> ans;
    if (m < n) {
      return ans;
    }
    vector<int> cnt1(26);
    for (char& c : p) {
      ++cnt1[c - 'a'];
    }
    vector<int> cnt2(26);
    for (int i = 0; i < n - 1; ++i) {
      ++cnt2[s[i] - 'a'];
    }
    for (int i = n - 1; i < m; ++i) {
      ++cnt2[s[i] - 'a'];
      if (cnt1 == cnt2) {
        ans.push_back(i - n + 1);
      }
      --cnt2[s[i - n + 1] - 'a'];
    }
    return ans;
  }
};
```

21 LC 28 | Find the Index of the First Occurrence in a String (Easy)

Problem: Return the zero-based index of the first occurrence of needle in haystack, or -1 if needle does not occur.
Constraints: 1 <= haystack.length, needle.length <= 10^4; haystack and needle consist of only lowercase English characters.

```
class Solution {
public:
  int strStr(string haystack, string needle) {
    int n = haystack.size(), m = needle.size();
    vector<int> nxt(m);
    for (int i = 1, j = 0; i < m; ++i) {
      while (j > 0 && needle[i] != needle[j]) {
        j = nxt[j - 1];
      }
      if (needle[i] == needle[j]) {
        ++j;
      }
      nxt[i] = j;
    }
    for (int i = 0, j = 0; i < n; ++i) {
      while (j > 0 && haystack[i] != needle[j]) {
        j = nxt[j - 1];
      }
      if (haystack[i] == needle[j]) {
        ++j;
      }
      if (j == m) {
        return i - m + 1;
      }
    }
    return -1;
  }
};
```

24 LC 5 | Longest Palindromic Substring (Medium)

Problem: Return a longest contiguous palindromic substring of s. Any longest answer is acceptable.
Constraints: 1 <= s.length <= 1000; s consist of only digits and English letters.

```
class Solution {
  vector<int> radii(const string& s) {
    int m = 2 * static_cast<int>(s.size()) + 1;
    vector<int> p(m);
    auto equal = [&](int a, int b) {
      return a % 2 == b % 2 && (a % 2 == 0 || s[a
        / 2] == s[b / 2]);
    };
    int center = 0, right = 0;
    for (int i = 0; i < m; ++i) {
      if (i < right) p[i] = min(right - i, p[2 *
        center - i]);
      while (i - p[i] - 1 >= 0 && i + p[i] + 1 < m
        &&
         equal(i - p[i] - 1, i + p[i] + 1))
          ++p[i];
      if (i + p[i] > right) { center = i; right =
        i + p[i]; }
    }
    return p;
  }
public:
  string longestPalindrome(string s) {
    auto p = radii(s);
    int center = ranges::max_element(p) -
      p.begin();
    return s.substr((center - p[center]) / 2,
      p[center]);
  }
};
```

23 LC 151 | Reverse Words in a String (Medium)

Problem: Return the words of s in reverse order. A word is a maximal sequence of non-space characters. Use one space between output words and omit leading/trailing spaces.
Constraints: 1 <= s.length <= 10^4; s contains English letters (upper-case and lower-case), digits, and spaces ' '.; There is at least one word in s.

```
class Solution {
public:
  string reverseWords(string s) {
    int i = 0;
    int j = 0;
    int n = s.size();
    while (i < n) {
      while (i < n && s[i] == ' ') {
        ++i;
      }
      if (i < n) {
        if (j != 0) {
          s[j++] = ' ';
        }
        int k = i;
        while (k < n && s[k] != ' ') {
          s[j++] = s[k++];
        }
        reverse(s.begin() + j - (k - i), s.begin()
          + j);
        i = k;
      }
    }
    s.erase(s.begin() + j, s.end());
    ranges::reverse(s);
    return s;
  }
};
```

25 LC 76 | Minimum Window Substring (Hard)

Problem: Return the shortest substring of s containing every character of t, including its required multiplicity. Return an empty string if none exists. The answer, when present, is unique.
Constraints: m == s.length; n == t.length; 1 <= m, n <= 10^5; s and t consist of uppercase and lowercase English letters.

```
class Solution {
public:
  string minWindow(string s, string t) {
    vector<int> need(128, 0);
    vector<int> window(128, 0);
    for (char c : t) {
      ++need[c];
    }
    int m = s.length(), n = t.length();
    int k = -1, mi = m + 1, cnt = 0;
    for (int l = 0, r = 0; r < m; ++r) {
      char c = s[r];
      if (++window[c] <= need[c]) {
        ++cnt;
      }
      while (cnt == n) {
        if (r - l + 1 < mi) {
          mi = r - l + 1;
          k = l;
        }
        c = s[l];
        if (window[c] <= need[c]) {
          --cnt;
        }
        --window[c];
        ++l;
      }
    }
    return k < 0 ? "" : s.substr(k, mi);
  }
};
```

27 LC 567 | Permutation in String (Medium)

Problem: Return whether s2 contains a contiguous substring that is a permutation of s1.
Constraints: 1 <= s1.length, s2.length <= 10^4; s1 and s2 consist of lowercase English letters.

```
class Solution {
public:
  bool checkInclusion(string s1, string s2) {
    int need = 0;
    int cnt[26]{};
    for (char c : s1) {
      if (++cnt[c - 'a'] == 1) {
        ++need;
      }
    }
    int m = s1.size(), n = s2.size();
    for (int i = 0; i < n; ++i) {
      int c = s2[i] - 'a';
      if (--cnt[c] == 0) {
        --need;
      }
      if (i >= m) {
        c = s2[i - m] - 'a';
        if (++cnt[c] == 1) {
          ++need;
        }
      }
      if (need == 0) {
        return true;
      }
    }
    return false;
  }
};
```

37 LC 36 | Valid Sudoku (Medium)

Problem: Check whether a partially filled 9-by-9 Sudoku board has no repeated digit in any row, column, or 3-by-3 box. Ignore '.' cells. Validity does not require that the board be solvable.
Constraints: board.length == 9; board[i].length == 9; board[i][j] is a digit 1-9 or '.'.

```
class Solution {
public:
  bool isValidSudoku(vector<vector<char>>& board)
    {
    vector<vector<bool>> row(9, vector<bool>(9,
      false));
    vector<vector<bool>> col(9, vector<bool>(9,
      false));
    vector<vector<bool>> sub(9, vector<bool>(9,
      false));
    for (int i = 0; i < 9; ++i) {
      for (int j = 0; j < 9; ++j) {
        char c = board[i][j];
        if (c == '.') continue;
        int num = c - '0' - 1;
        int k = i / 3 * 3 + j / 3;
        if (row[i][num] || col[j][num] ||
          sub[k][num]) {
          return false;
        }
        row[i][num] = true;
        col[j][num] = true;
        sub[k][num] = true;
      }
    }
    return true;
  }
};
```

31 LC 152 | Maximum Product Subarray (Medium)
Problem: Return the largest product of a nonempty contiguous subarray of nums. A one-element subarray has that element's value as its product.
Constraints: 1 <= nums.length <= 2 * 10^4; -10 <= nums[i] <= 10; The product of any subarray of nums is guaranteed to fit in a 32-bit integer.

```
class Solution {
public:
  int maxProduct(vector<int>& nums) {
    int f = nums[0], g = nums[0], ans = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
      int ff = f, gg = g;
      f = max({nums[i], ff * nums[i], gg *
        nums[i]});
      g = min({nums[i], ff * nums[i], gg *
        nums[i]});
      ans = max(ans, f);
    }
    return ans;
  }
};
```

38 LC 977 | Squares of a Sorted Array (Easy)
Problem: Given nondecreasing nums, return the squares of all its elements in nondecreasing order.
Constraints: 1 <= nums.length <= 10^4; -10^4 <= nums[i] <= 10^4; nums is sorted in non-decreasing order.

```
class Solution {
public:
  vector<int> sortedSquares(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n);
    for (int i = 0, j = n - 1, k = n - 1; i <= j;
      --k) {
      int a = nums[i] * nums[i];
      int b = nums[j] * nums[j];
      if (a > b) {
        ans[k] = a;
        ++i;
      } else {
        ans[k] = b;
        --j;
      }
    }
    return ans;
  }
};
```

48 LC 443 | String Compression (Medium)
Problem: Compress consecutive runs in chars in place. Write each character once, followed by its decimal run length only when the length exceeds 1. Return the written length; later entries are ignored. Use constant auxiliary space.
Constraints: 1 <= chars.length <= 2000; chars[i] is a lowercase English letter, uppercase English letter, digit, or symbol.

```
class Solution {
public:
  int compress(vector<char>& chars) {
    int k = 0, n = chars.size();
    for (int i = 0, j = i + 1; i < n;) {
      while (j < n && chars[j] == chars[i])
        ++j;
      chars[k++] = chars[i];
      if (j - i > 1) {
        for (char c : to_string(j - i)) {
          chars[k++] = c;
        }
      }
      i = j;
    }
    return k;
  }
};
```

49 LC 394 | Decode String (Medium)
Problem: Decode nested expressions k[text], which repeat text exactly k times. Repeat counts may have multiple digits. The encoding is valid, contains no extra spaces, and literal text contains no digits. Decoded length is at most 10^5.
Constraints: 1 <= s.length <= 30; s consists of lowercase English letters, digits, and square brackets '[]'.; s is guaranteed to be a valid input.; All the integers in s are in the range [1, 300].

```
class Solution {
public:
  string decodeString(string s) {
    vector<pair<string, int>> stack;
    string current;
    int count = 0;
    for (unsigned char c : s) {
      if (isdigit(c)) count = count * 10 + c -
        '0';
      else if (c == '[') {
        stack.emplace_back(move(current), count);
        current.clear();
        count = 0;
      } else if (c == ']') {
        auto [prefix, repeat] =
          move(stack.back());
        stack.pop_back();
        prefix.reserve(prefix.size() +
          current.size() * repeat);
        while (repeat--) prefix += current;
        current = move(prefix);
      } else current += static_cast<char>(c);
    }
    return current;
  }
};
```

50 LC 8 | String to Integer (atoi) (Medium)
Problem: Convert s manually to a signed 32-bit integer: skip leading spaces, read an optional sign, then consecutive decimal digits until the first non-digit or the end. Return 0 if no digits are read; clamp overflow to [-2^31, 2^31-1]. Do not use a built-in string-to-number conversion.
Constraints: 0 <= s.length <= 200; s consists of English letters (lower-case and upper-case), digits (0 - 9), ' ', '+', '-', and '.'.

```
class Solution {
public:
  int myAtoi(string s) {
    int i = 0, n = s.size();
    while (i < n && s[i] == ' ')
      ++i;
    int sign = 1;
    if (i < n && (s[i] == '-' || s[i] == '+')) {
      sign = s[i] == '-' ? -1 : 1;
      ++i;
    }
    int res = 0;
    while (i < n && isdigit(s[i])) {
      int digit = s[i] - '0';
      if (res > INT_MAX / 10 || (res == INT_MAX /
        10 && digit > INT_MAX % 10)) {
        return sign == 1 ? INT_MAX : INT_MIN;
      }
      res = res * 10 + digit;
      ++i;
    }
    return res * sign;
  }
};
```